HACKATHON BACHELOR DEV

IPSSI — Semaine du 2 au 6 mars 2026

SecureScan

Plateforme d'Analyse de Qualité et Sécurité de Code

Orchestrateur d'outils de sécurité open source

avec mapping OWASP Top 10 : 2025

Sujet du Hackathon

1. Contexte

Votre client, **CyberSafe Solutions**, est une startup française spécialisée en cyberstabilité pour les PME. Avec **150 entreprises clientes** et une équipe de 35 développeurs, CyberSafe développe des solutions de protection et d'audit pour ses clients.

Suite à un audit de l'ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information), plusieurs failles critiques ont été identifiées dans les projets internes de CyberSafe :

- Aucun outil centralisé d'analyse de sécurité du code source
- Vulnérabilités d'injection (SQL, XSS) non détectées avant mise en production
- Dépendances obsolètes avec des CVE connues dans plusieurs projets
- Aucune intégration de tests de sécurité dans les pipelines CI/CD
- Pas de rapport de sécurité automatisé pour les clients
- Corrections de sécurité manuelles, lentes et sources d'erreurs

CyberSafe Solutions vous mandate pour développer **SecureScan**, une plateforme web interne d'analyse de qualité et de sécurité de code. Cette plateforme doit orchestrer des outils de sécurité open source existants, agréger leurs résultats, les mapper sur l'OWASP Top 10 : 2025, et proposer des corrections automatisées.

2. Votre mission

Développer en une semaine la plateforme **SecureScan** qui répond aux besoins suivants :

2.1 Fonctionnalités obligatoires

A. Soumission de projet

- Saisie d'une URL de repository Git (GitHub / GitLab) ou upload d'une archive ZIP
- Clonage automatique du repository côté serveur
- Détection automatique du langage/framework (PHP, JavaScript, Node.js, Python…)

B. Analyse de sécurité automatisée

La plateforme doit intégrer et orchestrer au minimum 3 des outils suivants :

Type	Outils suggérés	Description
SAST	Semgrep, ESLint Security	Analyse statique du code source pour détecter des patterns vulnérables
Dépendances	npm audit, Composer audit	Scan des dépendances pour identifier les CVE connues
Secrets	git-secrets, TruffleHog	Détection de clés API, mots de passe et tokens dans le code
Qualité code	ESLint, PHPStan	Analyse de qualité générale et bonnes pratiques

Chaque outil doit être lancé via CLI, et sa sortie (JSON de préférence) parsée et stockée.

C. Mapping OWASP Top 10 : 2025

Chaque vulnérabilité détectée doit être classée selon le référentiel OWASP Top 10 : 2025 (dernière version en date) :

Rang	Catégorie	Exemples de détection
A01	Broken Access Control	IDOR, CORS mal configuré, escalade de privilèges
A02	Security Misconfiguration	Headers manquants, debug actif, config par défaut
A03	Software Supply Chain Failures	Dépendances vulnérables, packages malveillants
A04	Cryptographic Failures	Mots de passe en clair, algorithmes obsolètes
A05	Injection	SQL injection, XSS, command injection
A06	Insecure Design	Absence de validation, flux non sécurisés
A07	Authentication Failures	Brute force, sessions non invalidesées
A08	Software/Data Integrity Failures	CI/CD non sécurisé, désérialisation
A09	Logging & Alerting Failures	Logs absents, pas d'alertes sur erreurs
A10	Mishandling of Exceptional Conditions	Erreurs non gérées, fail-open, stack traces exposées

Votre outil doit couvrir au minimum 5 catégories sur 10.

D. Dashboard de visualisation

- Score de sécurité global (ex : A/B/C/D/F ou note sur 100)
- Répartition des vulnérabilités par sévérité (critique, haute, moyenne, basse)
- Distribution par catégorie OWASP Top 10 (graphique)
- Liste détaillée des findings avec : fichier concerné, ligne, description, sévérité, catégorie OWASP
- Filtres et tri (par sévérité, par outil source, par catégorie OWASP)

E. Système de correction automatisé

Pour les vulnérabilités les plus courantes, l'outil doit proposer des corrections prédéfinies (template-based) :

- Injection SQL → requêtes préparées / paramétrées
- XSS → échappement des sorties (htmlspecialchars, DOMPurify)
- Dépendances vulnérables → suggestion de version patchée
- Secrets exposés → remplacement par variable d'environnement
- Mots de passe en clair → hachage bcrypt/argon2

L'utilisateur peut valider ou rejeter chaque correction avant application.

F. Intégration Git automatisée

- Création automatique d'une branche de correction (ex : fix/securescan-2026-03-05)
- Application des corrections validées sur cette branche
- Push automatique via l'API Git (GitHub API / Octokit ou git CLI)
- Génération d'un rapport de sécurité (HTML ou PDF) résumant l'analyse et les corrections appliquées

2.2 Fonctionnalité BONUS — Correction intelligente par IA

Cette fonctionnalité est optionnelle et réservée aux groupes les plus avancés. Elle permettra de se démarquer lors de l'évaluation.

Au lieu de corrections template-based uniquement, intégrer un appel à une API LLM (OpenAI, Anthropic Claude, ou Mistral) pour :

- Analyser le contexte complet du code vulnérable
- Générer une correction adaptée au code existant (pas un template générique)
- Fournir une explication pédagogique de la vulnérabilité et du fix proposé
- Permettre à l'utilisateur de valider, modifier ou rejeter la suggestion IA avant application

Exemple d'intégration : envoyer le snippet de code vulnérable + le type de vulnérabilité OWASP à l'API, récupérer le code corrigé, l'afficher en diff côte-à-côte dans l'interface.

3. Technologies

Le choix des technologies est **libre**, dans la limite des langages et frameworks étudiés en cours. Suggestions :

- **Backend :** Node.js (Express), PHP (Symfony/Laravel), Python (Flask/FastAPI)
- **Frontend :** React, Vue.js, ou HTML/CSS/JS natif
- **Base de données :** MySQL, PostgreSQL, MongoDB, SQLite
- **Outils de sécurité :** Semgrep CLI, ESLint + eslint-plugin-security, npm audit, Composer audit, git-secrets, TruffleHog
- **Git API :** Octokit (GitHub API), GitLab API, ou git CLI (child_process/exec)
- **BONUS IA :** API OpenAI, API Anthropic Claude, API Mistral

4. Livrables attendus

Tous les livrables doivent être rendus le jeudi 5 mars à 17h00 au plus tard.

1. **Repository Git** avec historique de commits propre et README complet
2. **Maquettes / wireframes** de l'interface utilisateur
3. **Diagrammes UML** (cas d'utilisation, classes, activité, séquence)
4. **Application fonctionnelle** démontrant le workflow complet : soumission → analyse → résultats → correction → push
5. **Dashboard de visualisation** des résultats d'analyse avec mapping OWASP
6. **Rapport de sécurité généré** (exemple de sortie HTML ou PDF)
7. **Présentation PowerPoint** pour la soutenance du vendredi
8. **Documentation technique** (installation, configuration des outils, architecture)

5. Évaluation

L'évaluation porte sur deux volets : une **note technique /20 (coefficient 2)** et une **note de soutenance orale /20 (coefficient 1)**.

5.1 Grille technique (/20 — coefficient 2)

Critère	Points	Compétence
Repository Git (historique, README, .gitignore, branches)	**2**	Organisation
Architecture & intégration des outils de sécurité	**3**	Technique
Mapping OWASP Top 10 : 2025 (couverture et pertinence)	**4**	Sécurité
Système de correction automatisé (template-based + Git)	**4**	Développement
Dashboard & visualisation des résultats	**3**	Frontend
Projet fonctionnel et démonstration du workflow complet	**4**	Global

BONUS : +1 à +3 points pour l'intégration de la correction intelligente par IA (sur la note technique).

5.2 Grille de soutenance orale (/20 — coefficient 1)

Critère	Points	Catégorie
Démonstration de l'intégration des outils de sécurité	**1**	Technique
Interface utilisateur et expérience utilisateur	**2**	Frontend
Analyse et agrégation des résultats	**2**	Technique
Mapping OWASP Top 10 et classification	**2**	Sécurité
Système de fix automatisé (démonstration live)	**1**	Développement
Intégration Git API & gestion des branches	**1**	Technique
Génération de rapports de sécurité	**2**	Livrable
Démonstration complète du workflow	**2**	Global
Aisance à l'oral et réponses aux questions	**1.5**	Présentation
Qualité de la présentation (slides, structure)	**1.5**	Présentation

Projet final (impression générale, aboutissement)	**4**	Global

Moyenne finale = (Note technique × 2 + Note orale × 1) / 3

6. Planning de la semaine

Jour	Programme
Lundi 02/03	9h : Appel. 9h15-10h15 : Présentation générale du hackathon. 10h15 : Début du travail en autonomie. 14h : Appel. 16h : Désignation du team leader. 17h : Questions écrites pour le jury.
Mardi 03/03	9h : Appel. 9h15-10h15 : Session Q&A (jury + team leaders). Travail en autonomie. 14h : Appel. 17h : Questions pour le mercredi.
Mercredi 04/03	9h : Appel. 9h15-10h15 : Session Q&A. Travail en autonomie. 14h : Appel.
Jeudi 05/03	9h : Appel. Finalisation et préparation des soutenances. 14h : Appel. 17h00 : RENDU DES LIVRABLES (délai impératif).
Vendredi 06/03	9h : Appel. 9h30-12h : Soutenances (matin). 14h : Appel. 14h30-17h : Soutenances (après-midi). 20 min de présentation + 5-10 min de questions par groupe.

7. Sessions Questions / Réponses

Les sessions Q&A ont lieu le **mardi et mercredi matin de 9h15 à 10h15**. Fonctionnement :

- Les questions sont transmises la veille au soir (17h) par le team leader via l'encadrant
- Seules les questions sur le sujet et les consignes sont acceptées
- **Les questions techniques (code, bugs, architecture, configuration d'outils) ne seront PAS traitées**
- Les étudiants sont considérés comme des professionnels et doivent résoudre les problèmes techniques en autonomie

8. Ressources utiles

- **OWASP Top 10 : 2025 :** https://owasp.org/Top10/2025/
- **Semgrep :** https://semgrep.dev/docs/
- **ESLint Plugin Security :** https://github.com/eslint-community/eslint-plugin-security
- **npm audit :** https://docs.npmjs.com/cli/v6/commands/npm-audit
- **Octokit (GitHub API) :** https://github.com/octokit/octokit.js
- **TruffleHog :** https://github.com/trufflesecurity/trufflehog
- **OWASP Cheat Sheet Series :** https://cheatsheetseries.owasp.org/

Bon courage et bon hackathon !

À vous de sécuriser le code !